University of Sialkot

Web Engineering

Semester Project Report

Group Members

Abdullah Tanveer - 23010101-061

Muhammad Bilal - 23010101-079

BSSE -VI **Blue** **Fall 2023**

TOTAL GRADE POINT	**10**
GRADE POINT ALLOCATED	

ASSESSOR'S COMMENTS

Instructor: Museb Khalid

1. Introduction

Advanced Word Analyzer is a full-stack web application that lets a user paste or upload a piece of text and instantly receive a detailed statistical, structural, and AI-generated breakdown of it. It is built as a single-page Vue.js client backed by a Node.js/Express REST API, with MongoDB Atlas for persistent storage and Groq's LLM API for the AI-powered features.

The goal of the project was to combine deterministic text analysis (character counts, word frequency, reading time, etc.) with generative AI features (summarization, grammar checking, tone detection) in one clean interface, and to deploy it as a production-grade split-architecture application.

1.1 Objectives

- Provide instant, accurate statistical analysis of any block of text.
- Support direct text entry as well as file uploads (.txt and .docx).
- Layer AI-assisted writing feedback (summary, grammar, tone, improvements, difficult words, Q&A) on top of the raw analysis.
- Allow users to export their analysis as a polished PDF report.
- Collect lightweight user feedback for future iteration.
- Deploy the application with a production-ready, scalable architecture.

2. System Architecture

The application follows a decoupled client-server architecture. The Vue.js single-page application communicates with an independent Express REST API entirely over HTTP, and the API is the only component with direct access to MongoDB Atlas and the Groq AI service.

Layer	Description
Client	Vue 3 SPA (Vite build) — hosted on Vercel
Server	Express REST API — hosted on Vercel
Database	MongoDB Atlas (cloud-hosted MongoDB)
AI Provider	Groq API (Llama 3.3 70B Versatile)
Communication	JSON over HTTPS via Axios, CORS-restricted to the client domain

2.1 Request Flow

- Browser loads the static Vue.js build from Vercel's CDN.
- User actions (Analyze, Upload, Ask AI, etc.) trigger Axios calls from the client to the Vercel-hosted API, using a baseURL read from an environment variable (VITE_API_URL).
- The Express server validates the request, runs the relevant logic (local computation, file parsing, or a Groq API call), and optionally reads/writes MongoDB Atlas (feedback records).
- The server responds with JSON; the client updates its Pinia stores and re-renders the results reactively.

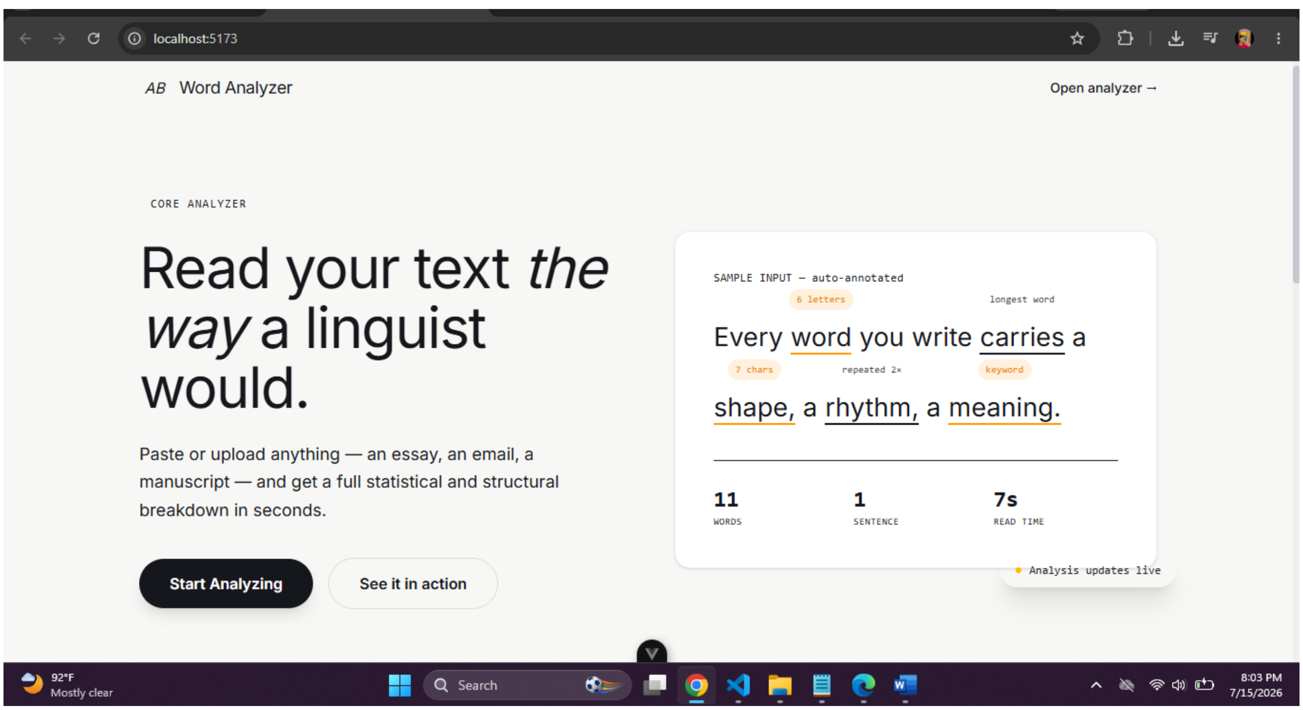
3. Technology Stack

Component	Technology
Frontend framework	Vue 3 (Composition API, TypeScript)
State management	Pinia (ai store, analyzer store)
Routing	Vue Router
Styling	Tailwind CSS
Charts	Chart.js via vue-chartjs (bar + doughnut charts)
Client-side PDF export	jsPDF + jspdf-autotable
HTTP client	Axios
Backend framework	Node.js + Express
Database & ODM	MongoDB Atlas + Mongoose
File upload handling	Multer (in-memory storage)
.docx text extraction	Mammoth
Server-side PDF export	PDFKit
AI integration	Groq SDK (llama-3.3-70b-versatile)
Client hosting	Vercel
Server hosting	Vercel

4. Core Features

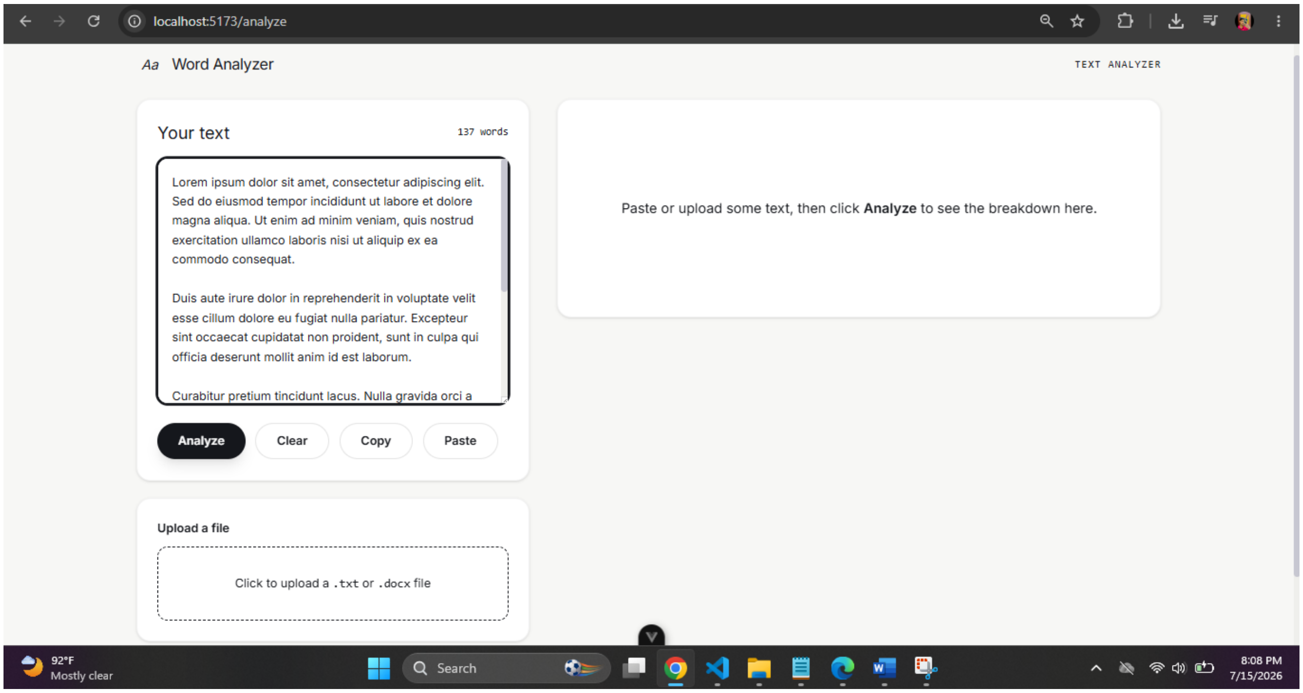
4.1 Home Page

A landing page introducing the analyzer, with a live annotated text sample and a call-to-action into the analyzer view.



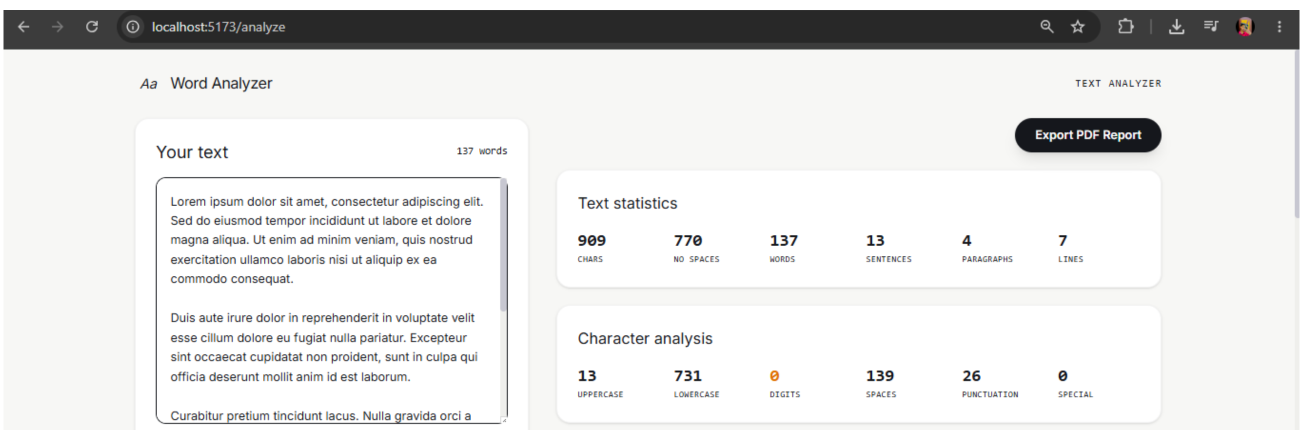
4.2 Text Input & File Upload

Users can type or paste text directly, or upload a .txt or .docx file (parsed server-side via Mammoth). A live word count updates as the user types.



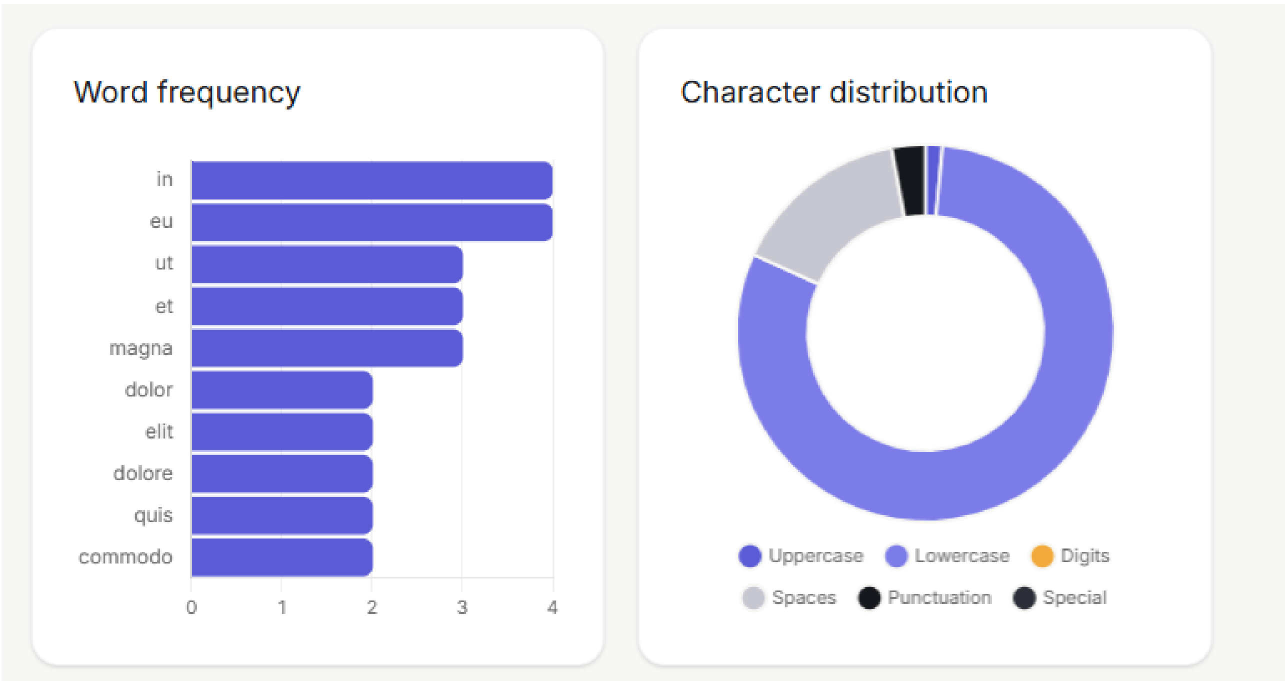
4.3 Text Statistics & Character Analysis

On clicking Analyze, the server computes character counts (uppercase, lowercase, digits, spaces, punctuation, special characters), and basic stats — total characters, words, sentences, paragraphs, and lines.



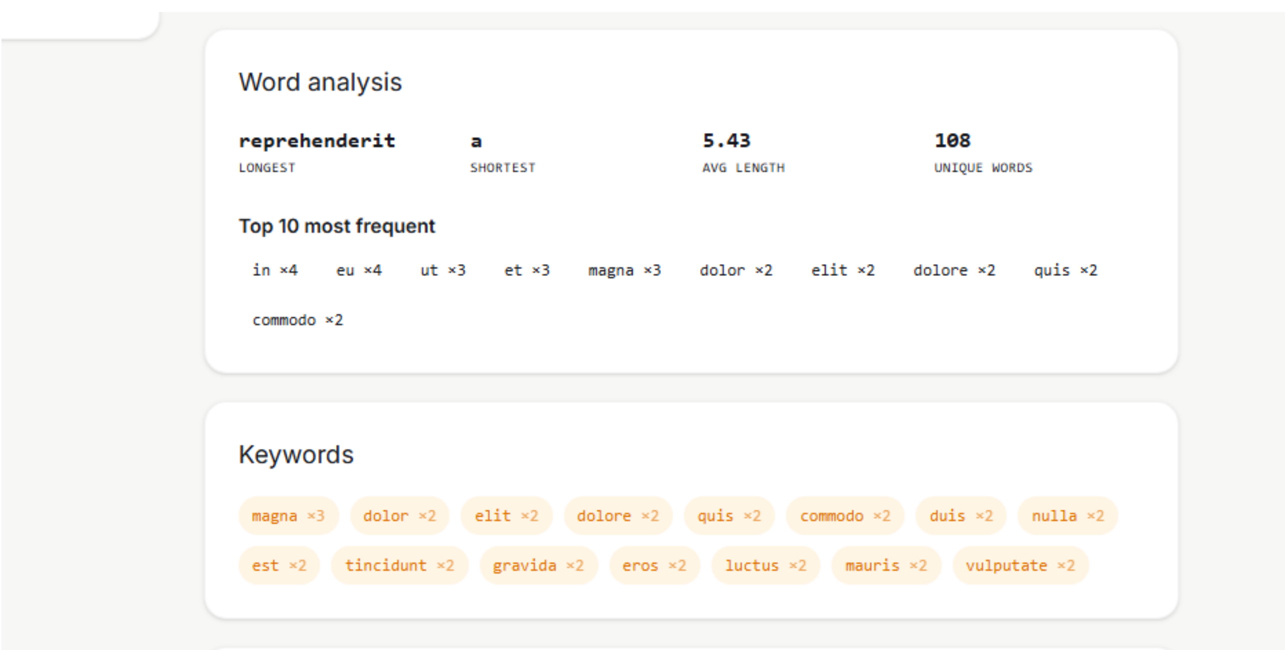
4.4 Word Frequency & Character Distribution Charts

Results are visualized with a horizontal bar chart of the most frequent words and a doughnut chart of character-type distribution, both rendered with Chart.js.



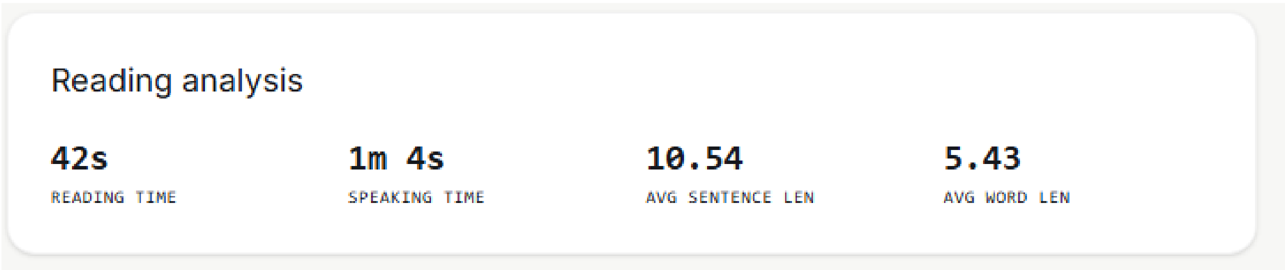
4.5 Word Analysis & Keyword Extraction

The app identifies the longest and shortest words, average word length, unique word count, and the top 10 most frequent words. A separate keyword extraction pass filters out common stop words to surface the terms that carry the most meaning.



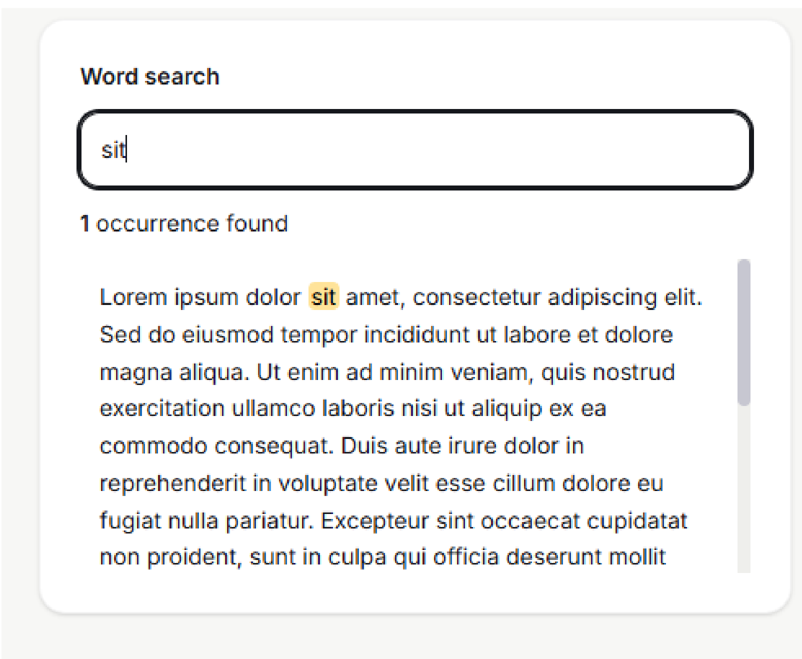
4.6 Reading & Speaking Time

Estimated reading time (200 wpm) and speaking time (130 wpm) are calculated, along with average sentence and word length.



4.7 Word Search & Highlight

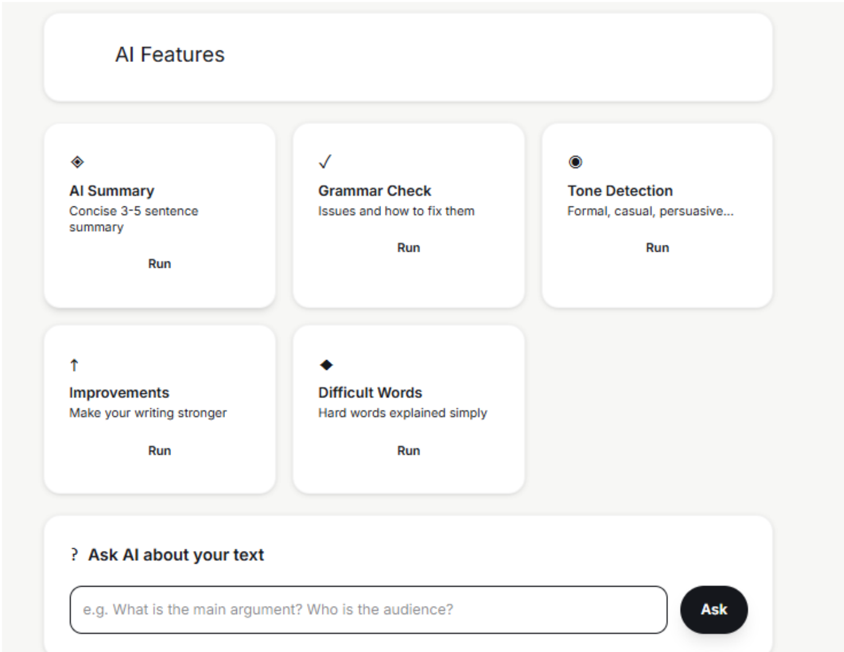
Users can search for a specific word within their text; matches are counted and highlighted inline.



4.8 AI Panel (Groq-Powered)

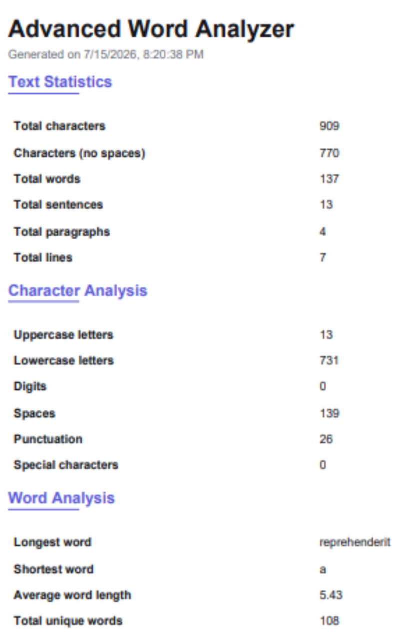
A dedicated AI panel offers five one-click features plus free-form Q&A, all powered by Groq's Llama 3.3 70B model:

- AI Summary — a concise 3-5 sentence summary of the text.
- Grammar Check — a list of grammar issues and how to fix them.
- Tone Detection — identifies the tone (formal, casual, persuasive, etc.) with a short justification.
- Improvements — 3-5 concrete suggestions to strengthen the writing.
- Difficult Words — explains advanced vocabulary in simple terms.
- Ask AI — free-form question answering scoped to the user's own text.



4.9 PDF Export

A full analysis report — statistics, character breakdown, word analysis, keywords, reading analysis, and rendered charts — can be exported as a PDF directly in the browser using jsPDF. A parallel server-side export route (PDFKit) is also available.



Advanced Word Analyzer

Generated on 7/15/2026, 8:20:38 PM

Text Statistics

Total characters	909
Characters (no spaces)	770
Total words	137
Total sentences	13
Total paragraphs	4
Total lines	7

Character Analysis

Uppercase letters	13
Lowercase letters	731
Digits	0
Spaces	139
Punctuation	26
Special characters	0

Word Analysis

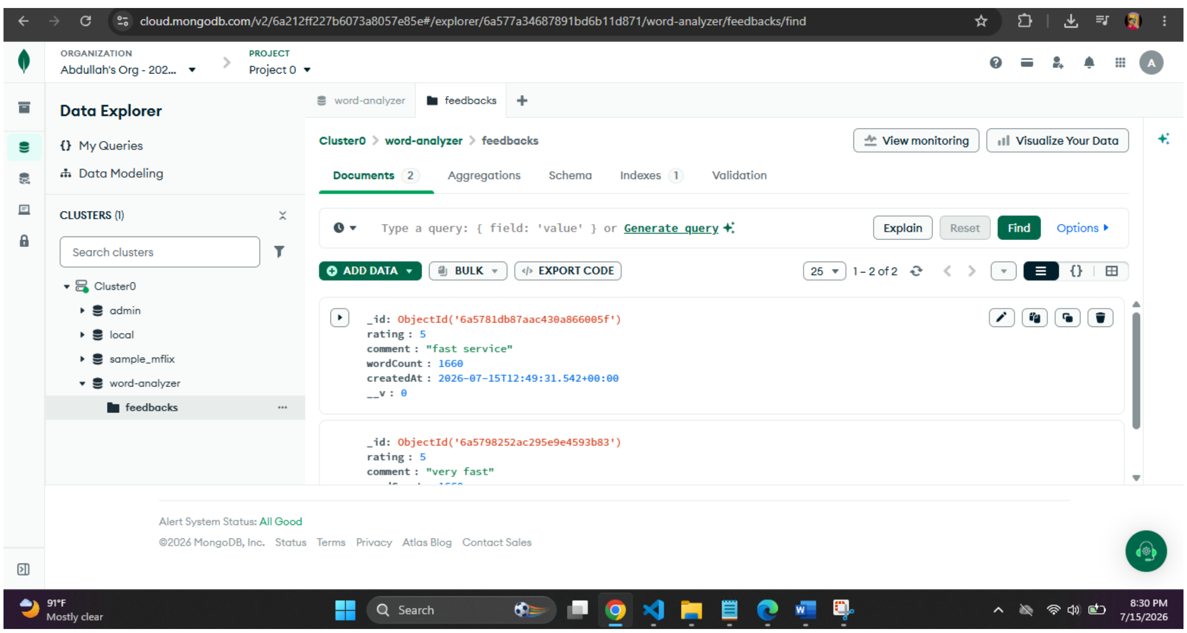
Longest word	reprehenderit
Shortest word	a
Average word length	5.43
Total unique words	108

4.10 Feedback Form

After viewing results, users can rate their experience (1-5 stars) with an optional comment. Feedback is stored in MongoDB Atlas via the /api/feedback endpoint for later review.

4.11 Database

All the feedbacks are successfully stored in mongodb altas in both development and deployment environment.



5. Application Flow

- 1. User lands on the Home page and clicks “Start Analyzing”, navigating to the Analyzer view (Vue Router).
- 2. User pastes text or uploads a .txt/.docx file — uploads are sent to POST /api/upload, parsed server-side, and the extracted text is loaded into the editor.
- 3. User clicks Analyze — the text is sent to POST /api/analyze, which runs character, word, keyword, and reading-time computations and returns a structured JSON result.
- 4. The Pinia analyzer store updates, and all result cards and charts re-render reactively.
- 5. User can optionally search for a word (client-side, instant), open the AI Panel to run any Groq-powered feature (POST /api/ai/*), export a PDF, or submit feedback (POST /api/feedback).
- 6. All AI requests and feedback writes go through the Express API, which is the only layer with credentials for Groq and MongoDB Atlas.

6. Deployment Architecture

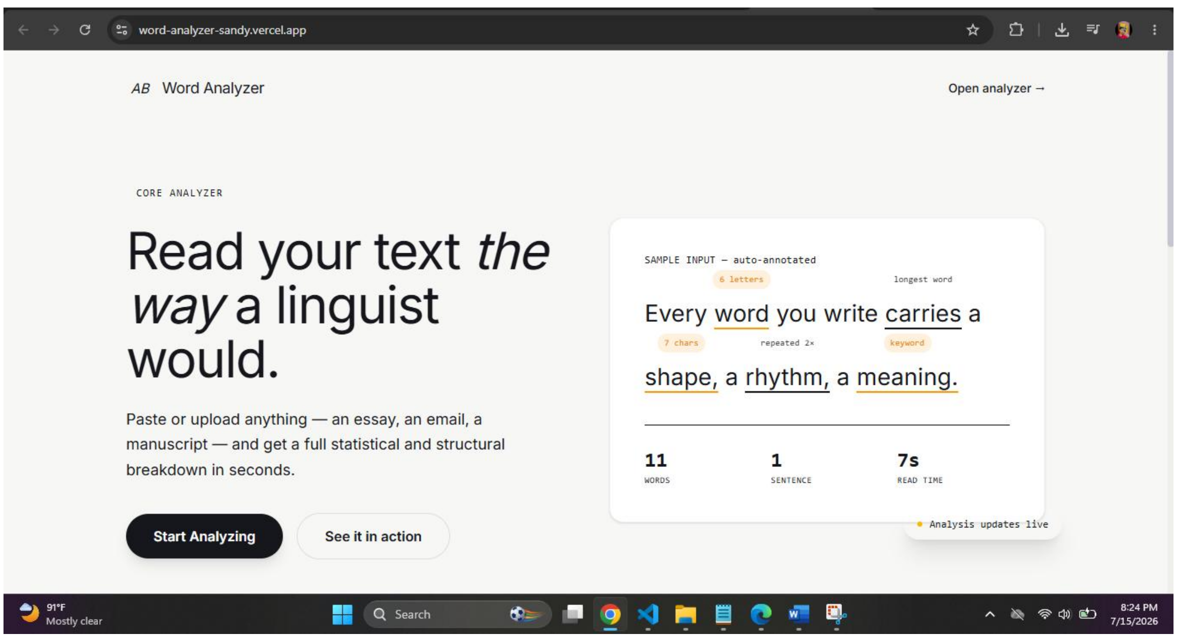
The client and server are deployed as two independent services rather than a single Vercel project, to avoid Vercel's serverless constraints on long-running/stateful Express connections.

Concern	Solution
Client (Vue/Vite build)	Deployed to Vercel; reads the API base URL from VITE_API_URL
Server (Express API)	Deployed to Vercel as a persistent Node service
Database	MongoDB Atlas, with network access whitelisted to 0.0.0.0/0 and the Mongoose connection cached to avoid exhausting the connection limit
CORS	Server restricts allowed origins to the deployed Vercel client domain via CLIENT_URL
Environment variables	MONGODB_URI, GROQ_API_KEY, CLIENT_URL set in Render; VITE_API_URL set in Vercel

6.1 Live Deployment

The application is live and fully functional, with both client and server served from Vercel .

https://word-analyzer-sandy.vercel.app/



7. Conclusion

Advanced Word Analyzer demonstrates a complete full-stack workflow: a reactive Vue 3 frontend, a modular Express API, cloud-hosted MongoDB storage, and integration with a third-party LLM provider for AI-assisted writing feedback. The split Vercel deployment reflects a production-realistic architecture that avoids the pitfalls of running a persistent database connection inside a serverless function. Future work could include user accounts, saved analysis history, and multi-language support.